Résolution et recherche par recuit simulé

Comme vu dans le cours le recuit simulé s'inspire de la thermodynamique dans le but de modéliser les systèmes physiques.

Objectif : Sortir d'un minimum local en acceptant avec une certaine probabilité une augmentation de la fonction objective (il s'agira de l'Energie E).

$$p = e^{-\frac{E_2 - E_1}{kT}}$$

k est la constante de Bolzmann, $k = 1,380510^{-23}$ J/K

T température absolue

Avec, p la probabilité qu'un système physique passe d'un niveau d'énergie E1 à un niveau d'énergie E2.

L'algorithme de recuit simulé consiste en fait à simuler le processus de refroidissement très lent en partant d'une température assez élevée. L'algorithme permet de placer la variable X au voisinage du minimum absolu, mais pas de manière certaine. Avec un bon réglage de la vitesse de décroissance de la température, elle permet d'obtenir le minimum absolu avec une forte probabilité.

On commence par ***une valeur X choisie aléatoirement*** dans l'intervalle [a, b]. La suite des états est générée par la suite des étapes inscrites dans le code suivant :

****

Choisir une température de départ T=T0

Une solution initiale X0 ; k=0

Un paramètre de tolérance Beta appartenant à [0,1]

Tant qu'on peut abaisser la température (1iere boucle

Tant que l'ensemble des voisins n'est pas vide

Générer une solution aléatoire dans le voisinage de la solution actuelle avec Xk+1appartient à N(Xk)

Calculer la variation de f cad delta f= f(Xk+1)-f(Xk)

3) Si deltaf<=0 (le cout diminue il ya amélioration, et Xk+1 est accepté

Xk+1=Xk

K=k+1

Sinon (Si deltaf >=0 , le cout augmente,

on calcule la probabilité d'acceptation p=exp(-delta(f)/T),

On choisit une valeur beta appartenant à [0,1]

Si ***p>beta,*** on accepte la nouvelle solution malgré qu'elle n'améliore pas la fonction cout ;

Xk+1=Xk,

K=k+1

Sinon
La solution est rejetée

Fin Si
Fin Si

Répéter 1,2, et 3 jusqu'à l'équilibre thermodynamique
Fin de tant que (boucle 2)

Fin de tant que (boucle 1)
****

- Choix d'un nouvel état x1=x+Δx ou x1=x-Δx. La direction est aléatoire mais le déplacement Δx est fixé au départ, assez petit par rapport à la largeur de l'intervalle.
- Si le nouvel état est plus probable que le précédent, c'est-à-dire si $f(x_1)<f(x)$, alors ce nouvel état est accepté.
- Si le nouvel état est moins probable que le précédent, il est accepté avec une probabilité égale au rapport des probabilités des deux états, c'est-à-dire exp(-(f(x1)-f(x))/T). Pour ce faire, on tire élatoirement un nombre réel avec une densité de probabilité uniforme sur l'intervalle *[0,1]* et on le compare à cette probabilité.

Dans une simulation de Metropolis d'un système physique, on fait des calculs statistiques avec la suite des états ainsi générée. Dans l'algorithme du recuit simulé, seule la dernière valeur de *x* est utilisée. L'itération est répétée un grand nombre de fois avec la même température, puis on recommence avec une température un peu plus faible, en partant de la dernière valeur de *x* obtenue avec la température précédente. Avec une température finale assez faible, l'état final obtenu est très probablement très proche du minimum absolu.

Cet algorithme nécessite des réglages qui dépendent du problème à résoudre : valeur de *Δx*, nombre d'itérations à chaque température, températures initiale et finale, loi de décroissance de la température.

Résolution et recherche par recherche de Tabou

Algorithme de recherche tabou

1 **Déterminer** une configuration aléatoire s
2 **Initialiser** une liste tabou vide
3 **tant que** le critère d'arrêt n'est pas satisfait **faire**
4 **Perturbation** de s suivant N mouvements non tabous
5 **Évaluation** des N voisins
6 **Sélection** du meilleur voisin t
7 **Actualisation** de la meilleure position connue s^*
8 **Insertion** du mouvement $t \rightarrow s$ dans la liste tabou
9 $s = t$
10 **fin**

ALGORITHME 1.3: Algorithme de recherche tabou.

La méthode Tabou est une méta-heuristique basée sur l'exploration de voisinage et permettant de sortir des optima locaux en acceptant des solutions non améliorantes. Afin d'éviter de boucler sur des solutions déjà visitées, elle garde en mémoire 'pendant un certain temps), la liste des solutions déjà visitées afin d'éviter de les considérer de nouveau.

Le principe général d'une méthode de tabou est le suivant :

- $s_{init} \leftarrow Glouton(Pb)$: // générer une solution initiale réalisable
- $s^* \leftarrow s_{init}$ // Mémoriser la meilleure solution
- $s \leftarrow s_{init}$ // solution courante
- $sTaboo \leftarrow \{s\}$ // solutions tabou
- $k \leftarrow 0$ // compteur itérations
- Répéter // Exploration des voisinages successifs
  1. $k \leftarrow k + 1$
  2. Choisir le meilleur voisin s' non tabou :
     - $s' \in Neighboor(s)$ tq $\forall s'' \in Neighboor(s), Obj(s') < Obj(s'')$ et $s' \notin sTaboo$
  3. Ajouter s' à $sTaboo$
  4. $s \leftarrow s'$
  5. si s' meilleur que s^* alors $s^* \leftarrow s'$
- Arrêt : $k \geq maxIter$ ou time out

from typing import List

```
import random

def voyageur_de_commerce (distances: List[List[int]], n: int, tabou_length: int, max_iter: int)
 - > List[int]:
   # Initialiser la liste tabou et la meilleure solution trouvée jusqu'à présent
   tabou_list = []
   best_solution = []
   best_distance = float("inf")
   # Initialise le compteur d'itérations
   iter_count = 0
      while True:
      # Générer une solution aléatoire
      solution = generate_random_solution(n)
       # Calculer la distance de la solution
      distance = calculate_distance(solution, distances)


       # Si la distance de la solution est meilleure que la meilleure solution trouvée jusqu'à présent,
      # mettre à jour la meilleure solution
      if distance < best_distance:
         best_solution = solution
         best_distance = distance
            # Ajouter la solution à la liste tabou
      tabou_list.append(solution)
            # Si la liste tabou a atteint sa longueur maximale, retirer le premier élément de la liste
      if len(tabou_list) > tabou_length:
         tabou_list.pop(0)
          # Incrémente le compteur d'itérations et vérifie si le nombre maximal d'itérations a été atteint
      iter_count += 1
```

```
        if iter_count >= max_iter:
            break
        return best_solution
    def generate_random_solution(n: int) -> List[int]:
    # Générer une solution aléatoire en mélangeant les villes
    cities = list(range(n))
    random.shuffle(cities)
    return cities
    def calculate_distance(solution: List[int], distances: List[List[int]]) -> int:
    # Calculer la distance totale de la solution en parcourant chaque ville
    distance = 0
    for i in range(len(solution) - 1):
        distance += distances[solution[i]][solution[i + 1]]
    return distance
# Créer la matrice de distances à partir des données fournies
distances = [
    [0, 12, 13, 15, 23, 25, 56, 27, 38, 29, 60, 32],
    [12, 0, 23, 22, 34, 56, 24, 78, 79, 34, 23, 24],
    [13, 23, 0, 25, 23, 19, 18, 42, 25, 23, 26, 33],
    [15, 22, 25, 0, 12, 24, 38, 27, 38, 49, 69, 29],
    [23, 34, 23, 12, 0, 54, 36, 71, 81, 19, 100, 22],
    [25, 56, 19, 24, 54, 0, 16, 27, 26, 37, 105, 72],
    [56, 24, 18, 38, 36, 16, 0, 18, 23, 24, 25, 26],
    [27, 78, 42, 27, 71, 27, 18, 0, 18, 19, 30, 53],
    [38, 79, 25, 38, 81, 26, 23, 18, 0, 29, 23, 22],
    [29, 34, 23, 49, 19, 37, 24, 19, 29, 0, 22, 42],
    [60, 23, 26, 69, 100, 105, 25, 30, 23, 22, 0, 36],
    [32, 24, 33, 29, 22, 72, 26, 53, 22, 42, 36, 0]]
print(voyageur_de_commerce(distances, 12, 12, 1000))
```